

CRE settlement workflow

How a Chainlink CRE workflow finds milestone claims whose dispute window has closed, and settles them without anyone pressing a button.

Runtime	Chainlink Runtime Environment (CRE), TypeScript SDK	
Trigger	Cron, every minute	
Chain	Base Sepolia (<code>ethereum-testnet-sepolia-base-1</code>)	
Code	<code>cre-workflow/settlement-workflow</code>	<code>contracts/src/automation</code>
Updated	September 2026	

1. Why it exists

A milestone claim in GrantLane is a UMA assertion with a dispute window. When the window closes without a dispute, the claim can be settled and the milestone paid. Settling is open to anyone, but somebody has to do it — otherwise an honest grantee waits for someone to press a button.

The settlement workflow is that somebody. Every minute it checks the escrow for claims whose window has closed and settles them.

Why CRE and not Chainlink Automation. The escrow was built for Automation — it implements `checkUpkeep` and `performUpkeep`. Automation's testnet service was sunset in June 2026, and its Base Sepolia registry no longer performs upkeeps. The workflow follows Chainlink's documented migration from Automation to CRE: it runs the same `checkUpkeep` and reaches the same `performUpkeep`, so the escrow needed no change.

A convenience, not a gate. The workflow can only do what any bystander can. If it stops, anyone can still settle from the app. It never decides an outcome — undisputed claims settle true because UMA says so, and disputed ones wait for their answer.

2. Where it sits

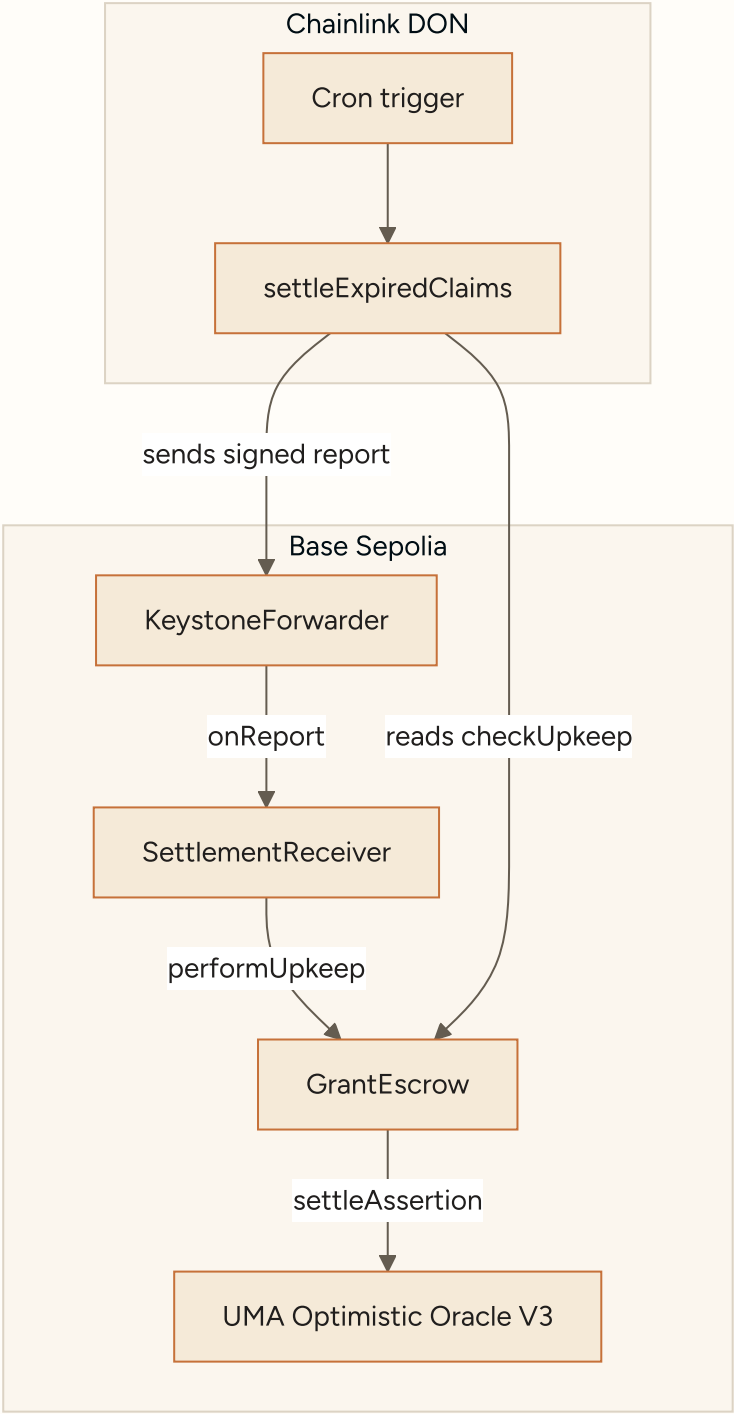
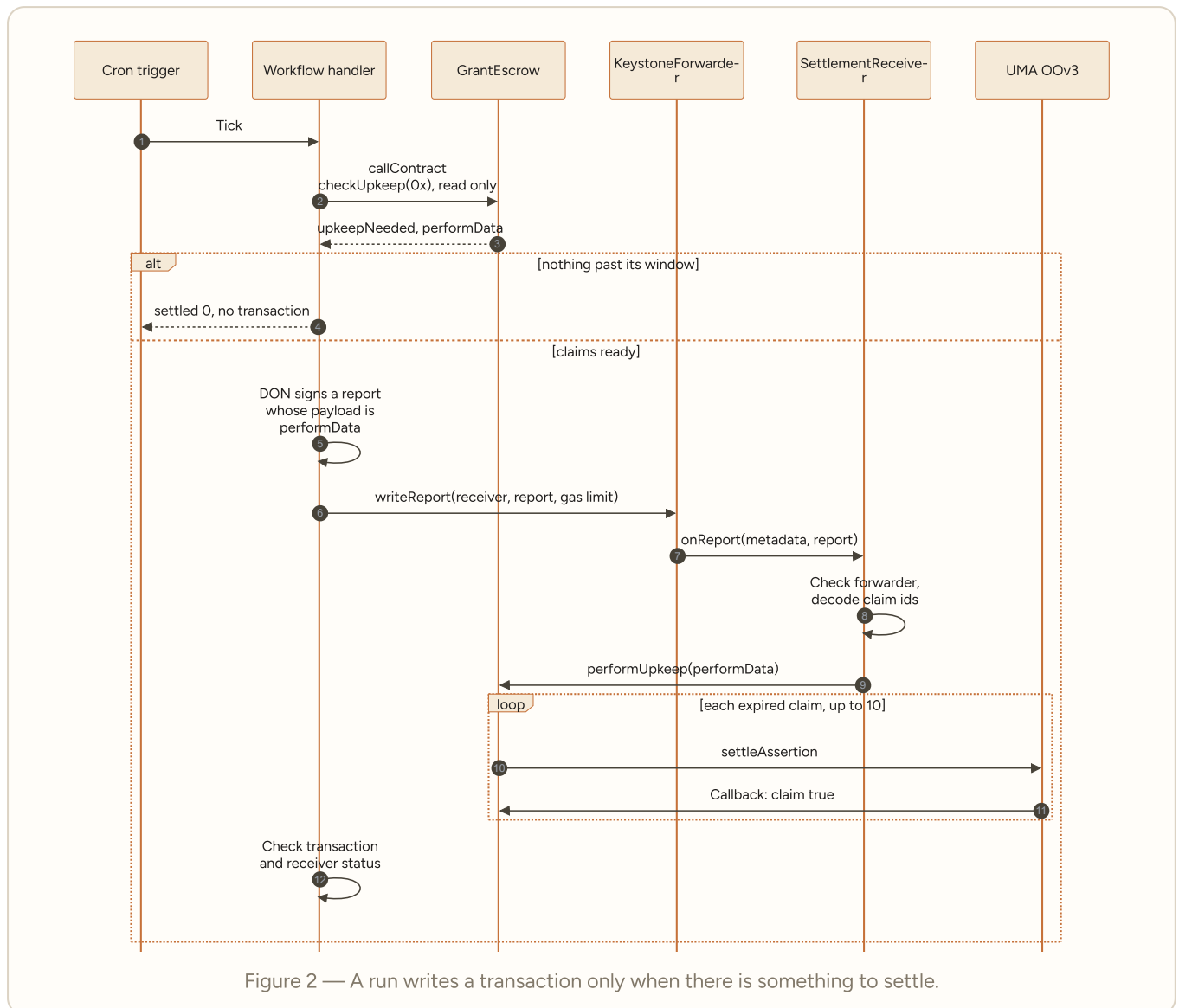


Figure 1 — The workflow reads off-chain and writes through Chainlink's forwarder into a receiver that can only call the escrow's upkeep.

Piece	Role
Cron trigger	Fires the handler on a schedule (<code>0 */1 * * * *</code> , every minute).

Workflow handler	<code>settleExpiredClaims</code> in <code>main.ts</code> : reads, decides whether to write, builds and sends the report.
KeystoneForwarder	Chainlink's contract that checks the DON's signatures on a report and delivers it to a receiver.
SettlementReceiver	Ours. Accepts reports only from the forwarder and passes them to <code>GrantEscrow.performUpkeep</code> . Nothing else.
GrantEscrow	Finds expired claims (<code>checkUpkeep</code>) and settles them on UMA (<code>performUpkeep</code>).

3. One run, step by step



- Read.** The handler calls `checkUpkeep` on the escrow with the EVM capability's `callContract`. It is a view call: no gas, no transaction.
- Decide.** If `upkeepNeeded` is false the run logs "No claims are past their dispute window" and ends.
- Report.** Otherwise the DON signs a report. Its payload is the escrow's own `performData`, passed through unchanged — the ABI-encoded list of claim ids.
- Write.** `writeReport` sends the report to the forwarder, addressed to the SettlementReceiver, with a 1,000,000 gas limit.
- Relay.** The forwarder verifies the signatures and calls `onReport`. The receiver checks the caller is the forwarder, decodes the ids and calls `performUpkeep`.
- Settle.** The escrow re-checks every id and settles each still-expired claim on UMA. UMA returns the bond and calls back; the escrow marks the milestone Approved and credits the payout.
- Confirm.** The handler checks both the transaction status and the receiver's execution status. The forwarder's transaction can succeed while the receiver reverts inside it, so both are required.

4. The on-chain half

GrantEscrow

Function	Behaviour
<code>checkUpkeep</code>	Scans the undisputed claims awaiting settlement and returns up to 10 whose window has closed, encoded as <code>performData</code> .
<code>performUpkeep</code>	Open to anyone. Decodes the ids, caps the batch at 10, and for each id skips it unless it is still awaiting settlement and expired. Each settlement runs in its own try/catch; a failure emits <code>AutoSettleFailed</code> and the batch continues.

Because `performUpkeep` re-validates everything it is given, stale or even invented input can do no harm: at worst it settles what anyone could already settle, or nothing.

SettlementReceiver

- Built on `ReceiverTemplate`: `onReport` rejects any caller other than the configured forwarder.
- Optionally rejects reports not written by an expected workflow owner or workflow name (`setExpectedAuthor`, `setExpectedWorkflowName`).
- Can only call `performUpkeep` on the one escrow it was deployed with. Chainlink's generic migration receiver forwards arbitrary calls; this one deliberately doesn't.
- Emits `SettlementRelayed(claims)` for every report it passes on.

Measured gas

A full batch of 10 settlements through the real UMA oracle measured about **581,000 gas**. The workflow uses a **1,000,000 gas** limit, leaving roughly 1.7× headroom. A fork test fails if a full batch ever exceeds it.

5. Files and configuration

```
cre-workflow/
├─ project.yaml          RPC endpoints per target
├─ .env.example          What the CRE CLI needs, documented
├─ settlement-workflow/
│   ├─ main.ts           The workflow handler
│   ├─ workflow.yaml     Workflow name and files per target
│   ├─ config.staging.json Used by simulation
│   ├─ config.production.json Used by a DON deployment
│   ├─ package.json      @chainlink/cre-sdk, viem
│   └─ tsconfig.json
└─

contracts/src/automation/
├─ SettlementReceiver.sol Report in, performUpkeep out
├─ ReceiverTemplate.sol   Forwarder and author checks
└─ IReceiver.sol
```

Workflow config

Field	Meaning
<code>schedule</code>	Cron expression with seconds. Every minute; CRE's minimum is 30 seconds.
<code>evm.chainSelectorName</code>	<code>ethereum-testnet-sepolia-base-1</code>
<code>evm.grantEscrowAddress</code>	The escrow whose <code>checkUpkeep</code> is read.
<code>evm.settlementReceiverAddress</code>	Where the report is delivered.
<code>evm.gasLimit</code>	Gas for the settlement transaction: 1,000,000.

Targets

Target	Used for	Forwarder
<code>staging-settings</code>	<code>cre workflow simulate</code> — runs locally; with <code>--broadcast</code> it sends a real transaction.	MockKeystoneForwarder <code>0x8230...f3e5</code> , which doesn't check DON signatures.
<code>production-settings</code>	<code>cre workflow deploy</code> — runs on a Chainlink DON.	KeystoneForwarder <code>0xF834...4482</code> .

The only secret the CLI needs is documented in `cre-workflow/.env.example`.

6. Running it

```
npm run workflow:typecheck    # type-check main.ts against the SDK
npm run workflow:simulate     # one run, read only: logs what it would settle
npm run workflow:broadcast    # one run that sends the settlement on Base Sepolia
```

A run with nothing to do prints:

```
[USER LOG] No claims are past their dispute window.
Workflow Simulation Result: {"settled":0}
```

A run that settles prints the claims and the transaction:

```
[USER LOG] Settling 1 claim(s): 0xff4c...613e
[USER LOG] Settlement relayed through SettlementReceiver: 0x1c00...7333
Workflow Simulation Result: {"settled":1,"txHash":"0x1c00...7333"}
```

Proven on Base Sepolia

Transaction `0x1c0040c9f0aeeec982ac2a1363283a960f201f4c439782de6ae99a73548407333` is a broadcast run: a `report` call to the `MockKeystoneForwarder`, which delivered it to the `SettlementReceiver`, which called `performUpkeep` and settled a live claim on UMA. The grantee then withdrew the payout.

7. Status and going live

Today: the workflow is built, tested and proven with real transactions, but it runs when started from the CRE CLI. Those runs are signed by our own wallet and pass through the mock forwarder. It is not yet autonomous, because deploying to a DON needs CRE deploy access, which has been requested.

Once deploy access is granted

1. **Point the receiver at the production forwarder** with `setForwarderAddress`.
2. **Restrict it to our workflow** with `setExpectedAuthor`, so only reports from our workflow owner are accepted.
3. **Deploy the workflow** with `cre workflow deploy` using `production-settings`.
4. **Check it:** make a claim and don't press Settle. About a minute after its window closes, the milestone should read Approved, settled by a transaction nobody on the team sent.

Details worth knowing

- The CRE SDK exports `TxStatus` but not `ReceiverContractExecutionStatus`; the workflow pins `SUCCESS = 0` from the protocol definition.
- The report payload is `performData` itself, so no extra encoding exists to get wrong.
- A simulated cron run executes once per command; only a deployed workflow runs on its schedule.